DESENVOLVIMENTO WEB COM .NET

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

RELATÓRIO DE AULAS PRÁTICAS

Relatório 6 – ASP.NET Core Identity, cookies, tokens bearer, JWT, roles, claims e rotas protegidas

Nome: _____

RA: _____

Polo de matrícula: _____

Local da realização da Aula Prática: _____

Ano da postagem: 2026

Instituto de Ciências Exatas e Tecnologia	**Disciplina:** Desenvolvimento Web com .NET **Título da Aula:** ASP.NET Core Identity, cookies, tokens bearer, JWT, roles, claims e rotas protegidas	**Relatório 6**

Unidade 6 – Autenticação e autorização ponta a ponta

Objetivo: Autenticar usuários, aplicar autorização por política e proteger operações e rotas do front-end sem expor credenciais ou tokens.

1. Identificação e ambiente

Repositório: https://github.com/maiaralopes-dev/projetos (branch `ccr-836bb896-v5ejij`, pasta `PortalTarefas/`); commit `6315a9c`. Pacote `Microsoft.AspNetCore.Identity.EntityFrameworkCore` 10.0.14; migration `IdentidadeEquipes` (tabelas `AspNet*`, coluna `Tarefas.Equipe` e tabela `EventosSeguranca`). As senhas e cookies que aparecem nas evidências foram **mascarados** (`********`, `<valor omitido>`); a senha dos usuários de teste existe só em User Secrets/`.env` locais e uma verificação automática confirmou que ela não aparece em `docs/`, `src/`, `tests/` nem `scripts/`.

2. Resumo teórico aplicado ao projeto

Autenticação × autorização. Autenticar é descobrir *quem* é o usuário (gera a **identidade**, um `ClaimsPrincipal`); autorizar é decidir *o que* ele pode fazer. **Claim** é uma afirmação sobre o usuário (`equipe=Backend`); **role** é uma claim especial de papel (`Usuario`, `Administrador`); **política** é um conjunto nomeado de requisitos. Quando falta autenticação o servidor faz **challenge** (401, ou redirecionamento para login no MVC); quando o usuário é conhecido mas não tem permissão faz **forbid** (403).

ASP.NET Core Identity gerencia usuários, *hash* de senhas (PBKDF2), papéis, claims, bloqueio por tentativas e o *security stamp*. Configuração: senha mínima de 10 caracteres com símbolo, bloqueio de 5 min após 5 falhas, e-mail único — nenhum controle foi desativado.

Tabela 1 – Trade-offs e decisão do esquema

	Cookie do Identity (escolhido)	**Token bearer do Identity**	**JWT de provedor OAuth/OIDC**
Origem/cliente	SPA e MVC na **mesma origem** que a API	Apps móveis/outros clientes da mesma API	Várias APIs/clientes, SSO, terceiros
Armazenamento	Cookie `HttpOnly` (JS não lê)	O cliente guarda o token (risco se em `localStorage`)	Idem; mais *refresh token*
CSRF	Exige proteção (SameSite + token antiforgery)	Não se aplica (cabeçalho manual)	Não se aplica
Revogação	Logout troca o *security stamp* → cookie copiado deixa de valer	Formato proprietário; expira; revogação limitada	Vida curta + revogação no provedor/refresh

Decisão: cookie, porque a SPA é servida pela própria API (proxy em dev, mesma origem em produção), o que evita expor qualquer token ao JavaScript. Controles: `HttpOnly`, `Secure`, `SameSite=Strict`, expiração de 30 min com renovação deslizante e **antiforgery** em toda operação que altera estado (cookie legível `XSRF-TOKEN` + cabeçalho `X-XSRF-TOKEN`, validado por um filtro de endpoint). Se surgirem clientes de outra origem, a recomendação é um provedor OIDC emitindo JWT de vida curta, sem gravar tokens em log.

401 × 403 e interface. Esconder um botão não protege nada: qualquer pessoa pode chamar a API com curl. Por isso **toda** regra é aplicada no servidor (inclusive nas ações MVC, outro caminho de entrada), e a SPA apenas reflete as permissões que o próprio servidor calcula em `GET /api/v2/auth/sessao`. **Rotas protegidas na SPA:** `RotaProtegida` redireciona para `/login` guardando a URL pretendida; a sessão é restaurada a cada carga consultando a API, já que o cookie é ilegível pelo JS.

3. Código-fonte e evidências

Tabela 2 – Políticas de autorização

Política	Requisito	Aplicada em
Fallback	Usuário autenticado (padrão seguro)	Tudo que não for `AllowAnonymous`
`LerTarefas`	Autenticado	`GET /api/v2/tarefas*`, interface MVC
`CriarTarefas`	Role `Usuario` ou `Administrador`	`POST /api/v2/tarefas`, MVC Create
`EditarTarefa` (por recurso)	Role + (mesma equipe **ou** claim `permissao=tarefas.editar-outras-equipes`)	`PUT` e MVC Edit/Concluir, via `IAuthorizationService` após carregar a tarefa
`EditarOutrasEquipes`	Claim `permissao=tarefas.editar-outras-equipes`	Usada pelo handler acima
`ExcluirTarefas`	Role `Administrador`	`DELETE`, detalhes de saúde, invalidação de cache

Usuários e roles são criados por **seed controlado** (`SementeUsuarios`): idempotente, com senha lida de `SeedUsuarios:SenhaInicial`; sem essa configuração nenhum usuário é criado. Perfis: ana (Usuario/Backend), bruno (Usuario/Frontend), carla (Administrador/Backend + claim de outras equipes) e visitante (sem papel). Login e logout: `POST /api/v2/auth/login` e `/logout` (API) e `ContaController` (MVC, mesmo cookie).

Tabela 3 – Matriz 401/403/200 — todas as células verificadas por teste automatizado; as principais também por curl

Operação (curl, fora da SPA)	Anônimo	visitante	ana (Backend)	bruno (Frontend)	carla (Admin)
Listar tarefas	401	200	200	200	200
Criar tarefa	401	403	201	201	201
Editar tarefa da equipe Backend (id 2)	401	403	200	403	200
Editar tarefa da equipe Frontend (id 8)	401	403	403	200	200
Excluir tarefa	401	403	403	403	204



Figura 1 – Ana (equipe Backend) vê a tarefa da equipe Frontend sem as ações de editar e excluir

4. Testes e validações obrigatórias

Tabela 4 – Testes obrigatórios (32 testes de integração .NET + 6 testes de componentes React)

Caso exigido	Resultado obtido	Evidência
Login válido e inválido sem revelar a conta	Senha errada e conta inexistente → mesmo 401 e mesmo texto; válido → 200 + cookie `secure; samesite=strict; httponly`	`02-login.txt`; Fig. B2
Logout e cookie invalidado	Logout 204; o cookie copiado **antes** do logout passa a receber 401	`04-logout.txt`; `Logout_invalida_cookie_copiado`
401 para não autenticado	API: 401 Problem Details; MVC: 302 para `/Conta/Login?ReturnUrl=...`	`03-matriz…`; teste
403 para autenticado sem permissão	visitante cria → 403; ana edita Frontend → 403; bruno exclui → 403	Tab. 3; 25 casos de matriz
Acesso por role/claim e reflexo na SPA	carla (claim) edita Frontend → 200; SPA oculta “Nova tarefa”, “Concluir” e “Excluir” conforme a sessão	Fig. 1, B4–B6

5. Aprimoramentos

- **Autorização por recurso** (`EditarTarefaHandler`): a decisão depende da equipe da tarefa, avaliada após carregá-la, igual na API e no MVC.
- **Auditoria de segurança** em `EventosSeguranca` (login com sucesso/falha, motivo, logout), sem senha, cookie ou token (`05-auditoria-e-usuarios.txt`).
- **Testes de integração** reais com cookie e antiforgery (cliente “navegador”), um esquema de autenticação de teste para a matriz e testes de bloqueio após 5 tentativas.
- **Defesas adicionais:** CSRF no login e nas mutações (sem token → 400), `LocalRedirect` contra redirecionamento aberto no MVC e rate limiting do login (unidade 7). Limitação conhecida: a diferença de tempo entre “conta inexistente” e “senha errada” não foi equalizada.

Referências

MICROSOFT. **Introduction to Identity on ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/authentication/identity. Acesso em: 9 nov. 2026.

MICROSOFT. **Choose an identity management solution**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/how-to-choose-identity-solution. Acesso em: 9 nov. 2026.

MICROSOFT. **Use Identity to secure a Web API backend for SPAs**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/authentication/identity-api-authorization. Acesso em: 9 nov. 2026.

MICROSOFT. **Policy-based authorization in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/authorization/policies. Acesso em: 9 nov. 2026.

MICROSOFT. **Resource-based authorization in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/authorization/resourcebased. Acesso em: 9 nov. 2026.

MICROSOFT. **Prevent Cross-Site Request Forgery (XSRF/CSRF) attacks in ASP.NET Core**. Microsoft Learn, 2025. Disponível em: https://learn.microsoft.com/aspnet/core/security/anti-request-forgery. Acesso em: 9 nov. 2026.

OWASP. **Session Management Cheat Sheet**. OWASP Cheat Sheet Series, 2025. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. Acesso em: 9 nov. 2026.

JONES, M.; BRADLEY, J.; SAKIMURA, N. **RFC 7519: JSON Web Token (JWT)**. IETF, 2015. Disponível em: https://www.rfc-editor.org/rfc/rfc7519. Acesso em: 9 nov. 2026.

Apêndice A – Código-fonte integral

Arquivo: `src/PortalTarefas.Web/Seguranca/Politicas.cs` (commit `6315a9c`)

```
using System.Security.Claims;
using Microsoft.AspNetCore.Authorization;
using PortalTarefas.Web.Modelos;

namespace PortalTarefas.Web.Seguranca;

public static class Papeis
{
    public const string Usuario = "Usuario";
    public const string Administrador = "Administrador";
}

public static class Claims
{
    public const string Equipe = "equipe";
    public const string Permissao = "permissao";
    public const string EditarOutrasEquipes = "tarefas.editar-outras-equipes";
}

public static class Politicas
{
    public const string LerTarefas = "LerTarefas";
    public const string CriarTarefas = "CriarTarefas";
    public const string EditarTarefa = "EditarTarefa";              // por recurso (equipe da tarefa)
    public const string EditarOutrasEquipes = "EditarOutrasEquipes"; // baseada em claim
    public const string ExcluirTarefas = "ExcluirTarefas";

    public static void Registrar(AuthorizationBuilder auth) => auth
        // Padrão seguro: tudo exige usuário autenticado, salvo o que for marcado com AllowAnonymous.
        .SetFallbackPolicy(new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build())
        .AddPolicy(LerTarefas, p => p.RequireAuthenticatedUser())
        .AddPolicy(CriarTarefas, p => p.RequireRole(Papeis.Usuario, Papeis.Administrador))
        .AddPolicy(EditarOutrasEquipes, p => p.RequireClaim(Claims.Permissao, Claims.EditarOutrasEquipes))
        .AddPolicy(EditarTarefa, p => p.AddRequirements(new EditarTarefaRequisito()))
        .AddPolicy(ExcluirTarefas, p => p.RequireRole(Papeis.Administrador));

    public static bool PodeEditarOutrasEquipes(this ClaimsPrincipal u) =>
        u.HasClaim(Claims.Permissao, Claims.EditarOutrasEquipes);

    public static string? Equipe(this ClaimsPrincipal u) => u.FindFirstValue(Claims.Equipe);
}

public sealed class EditarTarefaRequisito : IAuthorizationRequirement;

/// <summary>
/// Autorização por recurso: o usuário (com papel Usuario/Administrador) edita tarefas da própria equipe;
/// tarefas de outra equipe exigem a claim tarefas.editar-outras-equipes.
/// </summary>
public sealed class EditarTarefaHandler : AuthorizationHandler<EditarTarefaRequisito, Tarefa>
{
    protected override Task HandleRequirementAsync(AuthorizationHandlerContext ctx, EditarTarefaRequisito req,
Tarefa tarefa)
    {
        var u = ctx.User;
        var temPapel = u.IsInRole(Papeis.Usuario) || u.IsInRole(Papeis.Administrador);
        var mesmaEquipe = string.Equals(u.Equipe(), tarefa.Equipe, StringComparison.OrdinalIgnoreCase);
        if (temPapel && (mesmaEquipe || u.PodeEditarOutrasEquipes()))
            ctx.Succeed(req);
        return Task.CompletedTask;
    }
}
```

Arquivo: `src/PortalTarefas.Web/Seguranca/SementeUsuarios.cs` (commit `6315a9c`)

```
using System.Security.Claims;
using Microsoft.AspNetCore.Identity;

namespace PortalTarefas.Web.Seguranca;

/// <summary>
/// Cria papéis e usuários de teste de forma controlada. A senha inicial NÃO está no repositório:
/// vem de SeedUsuarios:SenhaInicial (User Secrets/variável de ambiente). Sem ela, nenhum usuário é criado.
/// </summary>
public static class SementeUsuarios
{
    private sealed record UsuarioTeste(string Email, string? Papel, string Equipe, string? Permissao = null);

    private static readonly UsuarioTeste[] Usuarios =
    [
        new("ana@portal.local", Papeis.Usuario, "Backend"),
        new("bruno@portal.local", Papeis.Usuario, "Frontend"),
        new("carla@portal.local", Papeis.Administrador, "Backend", Claims.EditarOutrasEquipes),
        new("visitante@portal.local", null, "Geral"),
    ];

    public static async Task ExecutarAsync(IServiceProvider sp, IConfiguration config, ILogger logger)
```

```
    {
        var roles = sp.GetRequiredService<RoleManager<IdentityRole>>();
        foreach (var papel in new[] { Papeis.Usuario, Papeis.Administrador })
            if (!await roles.RoleExistsAsync(papel)) await roles.CreateAsync(new IdentityRole(papel));

        var senha = config["SeedUsuarios:SenhaInicial"];
        if (string.IsNullOrWhiteSpace(senha))
        {
            logger.LogWarning("SeedUsuarios:SenhaInicial não configurada; usuários de teste não serão criados.");
            return;
        }

        var users = sp.GetRequiredService<UserManager<IdentityUser>>();
        foreach (var u in Usuarios)
        {
            if (await users.FindByEmailAsync(u.Email) is not null) continue;   // idempotente
            var novo = new IdentityUser { UserName = u.Email, Email = u.Email, EmailConfirmed = true };
            var r = await users.CreateAsync(novo, senha);
            if (!r.Succeeded)
                throw new InvalidOperationException("Falha ao criar usuário de teste: " + string.Join("; ",
r.Errors.Select(e => e.Code)));
            if (u.Papel is not null) await users.AddToRoleAsync(novo, u.Papel);
            await users.AddClaimAsync(novo, new Claim(Claims.Equipe, u.Equipe));
            if (u.Permissao is not null) await users.AddClaimAsync(novo, new Claim(Claims.Permissao,
u.Permissao));
            logger.LogInformation("Usuário de teste {Usuario} criado (papel {Papel}, equipe {Equipe}).", u.Email,
u.Papel ?? "nenhum", u.Equipe);
        }
    }
}
```

Arquivo: `src/PortalTarefas.Web/Seguranca/AntiforgeryApiFilter.cs` (commit `6315a9c`)

```
using Microsoft.AspNetCore.Antiforgery;
using Microsoft.AspNetCore.Identity;

namespace PortalTarefas.Web.Seguranca;

/// <summary>
/// Proteção CSRF da API autenticada por cookie: requisições que alteram estado (POST/PUT/PATCH/DELETE)
/// precisam do cabeçalho X-XSRF-TOKEN correspondente ao cookie antiforgery. Requisições autenticadas
/// por outro esquema (ex.: bearer em testes/clientes não-navegador) não dependem de cookie e são isentas.
/// </summary>
public sealed class AntiforgeryApiFilter(IAntiforgery antiforgery) : IEndpointFilter
{
    public async ValueTask<object?> InvokeAsync(EndpointFilterInvocationContext ctx, EndpointFilterDelegate next)
    {
        var http = ctx.HttpContext;
        var alteraEstado = !(HttpMethods.IsGet(http.Request.Method) || HttpMethods.IsHead(http.Request.Method) ||
HttpMethods.IsOptions(http.Request.Method));
        var tipoAuth = http.User.Identity?.AuthenticationType;
        var usaCookieOuAnonimo = tipoAuth is null || tipoAuth == IdentityConstants.ApplicationScheme;

        if (alteraEstado && usaCookieOuAnonimo && !await antiforgery.IsRequestValidAsync(http))
            return TypedResults.Problem(statusCode: 400, title: "Token antiforgery ausente ou inválido.",
                detail: "Obtenha a sessão (GET /api/v2/auth/sessao) e envie o cabeçalho X-XSRF-TOKEN.");
        return await next(ctx);
    }
}

public static class AntiforgeryExtensions
{
    public const string CookieLegivel = "XSRF-TOKEN";

    /// <summary>Gera o par de tokens e grava o token de requisição num cookie legível pelo JavaScript da
SPA.</summary>
    public static void EmitirTokenAntiforgery(this HttpContext http, IAntiforgery antiforgery)
    {
        var tokens = antiforgery.GetAndStoreTokens(http);
        http.Response.Cookies.Append(CookieLegivel, tokens.RequestToken!, new CookieOptions
        {
            HttpOnly = false, Secure = true, SameSite = SameSiteMode.Strict, Path = "/"
        });
    }
}
```

Arquivo: `src/PortalTarefas.Web/Endpoints/AuthEndpoints.cs` (commit `6315a9c`)

```
using System.Security.Claims;
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Antiforgery;
using Microsoft.AspNetCore.Http.HttpResults;
using Microsoft.AspNetCore.Identity;
using PortalTarefas.Web.Dados;
using PortalTarefas.Web.Modelos;
using PortalTarefas.Web.Seguranca;

namespace PortalTarefas.Web.Endpoints;

public sealed record LoginDto(
```

```
    [property: Required(ErrorMessage = "Informe o e-mail."), EmailAddress(ErrorMessage = "E-mail inválido.")]
string Email,
    [property: Required(ErrorMessage = "Informe a senha.")] string Senha);

/// <summary>Sessão atual e permissões calculadas no servidor (a SPA apenas as reflete na interface).</summary>
public sealed record SessaoDto(bool Autenticado, string? Usuario, string? Equipe, IReadOnlyList<string> Papeis,
    bool PodeCriar, bool PodeExcluir, bool PodeEditarOutrasEquipes);

public static class AuthEndpoints
{
    public static RouteGroupBuilder MapAuthEndpoints(this IEndpointRouteBuilder app)
    {
        var g = app.MapGroup("/api/v2/auth").WithTags("Autenticação");

        g.MapGet("/sessao", Sessao).AllowAnonymous().WithName("ObterSessao")
            .WithSummary("Retorna a sessão atual e emite o token antiforgery (cookie XSRF-TOKEN).");
        g.MapPost("/login", Login).AllowAnonymous().WithName("Entrar")
            .WithSummary("Autentica com e-mail e senha e emite o cookie de sessão (HttpOnly, Secure,
SameSite=Strict).")
            .ProducesProblem(StatusCodes.Status401Unauthorized);
        g.MapPost("/logout", Logout).WithName("Sair")
            .WithSummary("Encerra a sessão e invalida cookies emitidos anteriormente (troca do security stamp).")
            .ProducesProblem(StatusCodes.Status401Unauthorized);
        return g;
    }

    internal static Ok<SessaoDto> Sessao(HttpContext http, IAntiforgery antiforgery)
    {
        http.EmitirTokenAntiforgery(antiforgery);
        return TypedResults.Ok(CriarSessao(http.User));
    }

    internal static async Task<Results<Ok<SessaoDto>, ProblemHttpResult>> Login(
        LoginDto dto, SignInManager<IdentityUser> signIn, IAntiforgery antiforgery, PortalTarefasDbContext db,
        HttpContext http, ILogger<LoginDto> logger)
    {
        var usuario = await signIn.UserManager.FindByEmailAsync(dto.Email);
        var resultado = usuario is null
            ? SignInResult.Failed
            : await signIn.PasswordSignInAsync(usuario, dto.Senha, isPersistent: false, lockoutOnFailure: true);

        db.EventosSeguranca.Add(new EventoSeguranca
        {
            Evento = "Login", Usuario = dto.Email.Trim().ToLowerInvariant(), Sucesso = resultado.Succeeded,
            Detalhe = resultado.Succeeded ? null : resultado.IsLockedOut ? "bloqueado" : usuario is null ? "conta
inexistente" : "senha incorreta"
        });
        await db.SaveChangesAsync();

        if (!resultado.Succeeded)
        {
            logger.LogWarning("Falha de login (bloqueado: {Bloqueado})", resultado.IsLockedOut);
            // Mesma resposta para conta inexistente, senha errada ou bloqueio: não revela se a conta existe.
            return TypedResults.Problem(statusCode: 401, title: "Não foi possível entrar.",
                detail: "E-mail ou senha inválidos, ou acesso temporariamente bloqueado.");
        }

        // O token antiforgery é vinculado à identidade: após o login é preciso emitir um novo.
        http.User = await signIn.CreateUserPrincipalAsync(usuario!);
        http.EmitirTokenAntiforgery(antiforgery);
        return TypedResults.Ok(CriarSessao(http.User));
    }

    internal static async Task<NoContent> Logout(SignInManager<IdentityUser> signIn, PortalTarefasDbContext db,
HttpContext http)
    {
        if (await signIn.UserManager.GetUserAsync(http.User) is { } usuario)
            await signIn.UserManager.UpdateSecurityStampAsync(usuario);   // cookies antigos deixam de valer
        await signIn.SignOutAsync();
        db.EventosSeguranca.Add(new EventoSeguranca { Evento = "Logout", Usuario = http.User.Identity?.Name,
Sucesso = true });
        await db.SaveChangesAsync();
        http.Response.Cookies.Delete(AntiforgeryExtensions.CookieLegivel);
        return TypedResults.NoContent();
    }

    public static SessaoDto CriarSessao(ClaimsPrincipal u) => u.Identity?.IsAuthenticated == true
        ? new SessaoDto(true, u.Identity.Name, u.Equipe(),
            u.FindAll(ClaimTypes.Role).Select(c => c.Value).ToList(),
            u.IsInRole(Papeis.Usuario) || u.IsInRole(Papeis.Administrador),
            u.IsInRole(Papeis.Administrador),
            u.PodeEditarOutrasEquipes())
        : new SessaoDto(false, null, null, [], false, false, false);
}
```

Arquivo: `src/PortalTarefas.Web/Controllers/ContaController.cs` (commit `87ccb46`)

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
```

```
namespace PortalTarefas.Web.Controllers;

public sealed class LoginViewModel
{
    [Required(ErrorMessage = "Informe o e-mail."), EmailAddress, Display(Name = "E-mail")]
    public string Email { get; set; } = "";

    [Required(ErrorMessage = "Informe a senha."), DataType(DataType.Password), Display(Name = "Senha")]
    public string Senha { get; set; } = "";
}

/// <summary>Login/logout da interface MVC (mesmo cookie de sessão usado pela SPA).</summary>
public sealed class ContaController(SignInManager<IdentityUser> signIn) : Controller
{
    [HttpGet, AllowAnonymous]
    public IActionResult Login(string? returnUrl) { ViewData["ReturnUrl"] = returnUrl; return View(new
LoginViewModel()); }

    [HttpPost, AllowAnonymous]
    public async Task<IActionResult> Login(LoginViewModel m, string? returnUrl)
    {
        ViewData["ReturnUrl"] = returnUrl;
        if (!ModelState.IsValid) return View(m);
        var usuario = await signIn.UserManager.FindByEmailAsync(m.Email);
        var r = usuario is null ? Microsoft.AspNetCore.Identity.SignInResult.Failed : await
signIn.PasswordSignInAsync(usuario, m.Senha, false, lockoutOnFailure: true);
        if (!r.Succeeded)
        {
            ModelState.AddModelError("", "E-mail ou senha inválidos, ou acesso temporariamente bloqueado.");
            return View(m);
        }
        // LocalRedirect impede redirecionamento aberto para outro site.
        return Url.IsLocalUrl(returnUrl) ? LocalRedirect(returnUrl) : RedirectToAction("Index", "Tarefas");
    }

    [HttpPost, Authorize]
    public async Task<IActionResult> Logout()
    {
        if (await signIn.UserManager.GetUserAsync(User) is { } u) await
signIn.UserManager.UpdateSecurityStampAsync(u);
        await signIn.SignOutAsync();
        return RedirectToAction(nameof(Login));
    }

    [HttpGet, AllowAnonymous]
    public IActionResult AcessoNegado() { Response.StatusCode = 403; return View(); }
}
```

Arquivo: `src/PortalTarefas.Frontend/src/auth/AuthContext.tsx` (commit `6315a9c`)

```
import { createContext, useCallback, useContext, useEffect, useMemo, useState, type ReactNode } from 'react'
import { chamar, registrarAoNaoAutenticado, type ErroApi, type SessaoDto } from '../api/http'

const anonima: SessaoDto = { autenticado: false, usuario: undefined, equipe: undefined, papeis: [], podeCriar:
false, podeExcluir: false, podeEditarOutrasEquipes: false }

export interface Auth {
  carregando: boolean
  sessao: SessaoDto
  entrar(email: string, senha: string): Promise<ErroApi | null>
  sair(): Promise<void>
  /** Reflexo, na interface, da mesma regra aplicada pelo servidor (que continua sendo a autoridade). */
  podeEditar(equipeDaTarefa: string): boolean
}

export const AuthContext = createContext<Auth | null>(null)

export function AuthProvider({ children }: { children: ReactNode }) {
  const [sessao, setSessao] = useState<SessaoDto>(anonima)
  const [carregando, setCarregando] = useState(true)

  // Restaura a sessão ao abrir/recarregar a SPA: o cookie HttpOnly não é legível, então pergunta à API.
  const restaurar = useCallback(async () => {
    const r = await chamar((c, s) => c.obterSessao(s))
    setSessao(r.ok ? r.dados : anonima)
    setCarregando(false)
  }, [])

  useEffect(() => { restaurar() }, [restaurar])
  useEffect(() => {
    registrarAoNaoAutenticado(() => setSessao(anonima))   // 401 em qualquer chamada => sessão encerrada
    return () => registrarAoNaoAutenticado(undefined)
  }, [])

  const valor = useMemo<Auth>(() => ({
    carregando,
    sessao,
    async entrar(email, senha) {
      const r = await chamar((c, s) => c.entrar({ email, senha }, s))
      if (!r.ok) return r.erro
```

```
      setSessao(r.dados)
      return null
    },
    async sair() {
      await chamar((c, s) => c.sair(s))
      setSessao(anonima)
      await restaurar()   // obtém novo token antiforgery anônimo
    },
    podeEditar(equipe) {
      return sessao.podeCriar && (sessao.equipe === equipe || sessao.podeEditarOutrasEquipes)
    },
  }), [carregando, sessao, restaurar])

  return <AuthContext.Provider value={valor}>{children}</AuthContext.Provider>
}

export function useAuth(): Auth {
  const ctx = useContext(AuthContext)
  if (!ctx) throw new Error('useAuth deve ser usado dentro de <AuthProvider>')
  return ctx
}
```

Arquivo: `src/PortalTarefas.Frontend/src/auth/RotaProtegida.tsx` (commit `6315a9c`)

```
import type { ReactNode } from 'react'
import { Navigate, useLocation } from 'react-router'
import { Carregando } from '../components/Comuns'
import { useAuth } from './AuthContext'

/**
 * Protege uma rota da SPA: sem sessão, redireciona para /login guardando a URL pretendida.
 * É só experiência do usuário — a API valida tudo de novo e responde 401/403.
 */
export function RotaProtegida({ children, exigeCriar = false }: { children: ReactNode; exigeCriar?: boolean }) {
  const { carregando, sessao } = useAuth()
  const local = useLocation()
  if (carregando) return <Carregando texto="Verificando sessão…" />
  if (!sessao.autenticado) return <Navigate to="/login" replace state={{ de: local.pathname + local.search }} />
  if (exigeCriar && !sessao.podeCriar)
    return <div className="estado erro" role="alert"><strong>Sem permissão</strong><p>Seu perfil não pode
cadastrar tarefas.</p></div>
  return children
}
```

Arquivo: `src/PortalTarefas.Frontend/src/pages/Login.tsx` (commit `6315a9c`)

```
import { useState, type FormEvent } from 'react'
import { Navigate, useLocation, useNavigate } from 'react-router'
import type { ErroApi } from '../api/http'
import { useAuth } from '../auth/AuthContext'
import { MensagemErro, TituloPagina } from '../components/Comuns'

export function Login() {
  const { entrar, sessao } = useAuth()
  const navegar = useNavigate()
  const destino = (useLocation().state as { de?: string } | null)?.de ?? '/tarefas'
  const [erro, setErro] = useState<ErroApi | null>(null)
  const [enviando, setEnviando] = useState(false)

  if (sessao.autenticado) return <Navigate to={destino} replace />

  async function enviar(e: FormEvent<HTMLFormElement>) {
    e.preventDefault()
    const f = new FormData(e.currentTarget)
    setEnviando(true)
    const falha = await entrar(String(f.get('email')), String(f.get('senha')))
    setEnviando(false)
    if (falha) setErro(falha)
    else navegar(destino, { replace: true })   // volta para a URL que o usuário tentou abrir
  }

  return (
    <>
      <TituloPagina>Entrar</TituloPagina>
      <form onSubmit={enviar} className="formulario login" noValidate>
        <label>E-mail<input name="email" type="email" autoComplete="username" required /></label>
        <label>Senha<input name="senha" type="password" autoComplete="current-password" required /></label>
        {erro && <MensagemErro erro={erro} />}
        <button type="submit" disabled={enviando}>{enviando ? 'Entrando…' : 'Entrar'}</button>
      </form>
    </>
  )
}
```

Configuração de autenticação no Program.cs

```
// Program.cs (trecho): Identity + cookie
builder.Services.AddAuthentication(IdentityConstants.ApplicationScheme).AddIdentityCookies();
```

```
builder.Services.AddIdentityCore<IdentityUser>(o => { o.Password.RequiredLength = 10;
o.Password.RequireNonAlphanumeric = true;
        o.Lockout.MaxFailedAccessAttempts = 5; o.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
o.User.RequireUniqueEmail = true; })
    .AddRoles<IdentityRole>().AddEntityFrameworkStores<PortalTarefasDbContext>().AddSignInManager().AddDefaultTok
enProviders();
builder.Services.ConfigureApplicationCookie(o => {
    o.Cookie.Name = "portal.sessao"; o.Cookie.HttpOnly = true; o.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    o.Cookie.SameSite = SameSiteMode.Strict; o.ExpireTimeSpan = TimeSpan.FromMinutes(30); o.SlidingExpiration =
true;
    o.LoginPath = "/Conta/Login"; o.AccessDeniedPath = "/Conta/AcessoNegado";
    o.Events.OnRedirectToLogin = ctx => ResponderOuRedirecionar(ctx, 401);          // /api -> 401
    o.Events.OnRedirectToAccessDenied = ctx => ResponderOuRedirecionar(ctx, 403);   // /api -> 403
});
builder.Services.Configure<SecurityStampValidatorOptions>(o => o.ValidationInterval = TimeSpan.Zero);
Politicas.Registrar(builder.Services.AddAuthorizationBuilder());
builder.Services.AddSingleton<IAuthorizationHandler, EditarTarefaHandler>();
builder.Services.AddAntiforgery(o => { o.HeaderName = "X-XSRF-TOKEN"; o.Cookie.SecurePolicy =
CookieSecurePolicy.Always; o.Cookie.SameSite = SameSiteMode.Strict; });
...
app.UseAuthentication();
app.UseAuthorization();
```

Apêndice B – Evidências de execução

Login: sem antiforgery, senha errada, conta inexistente e válido (valores mascarados)

```
### Login sem token antiforgery (CSRF)
$ curl -b /tmp/tmp.qzHMclHn3e/anon -X POST -H Content-Type: application/json -d
{"email":"ana@portal.local","senha":"********"} https://localhost:7080/api/v2/auth/login
HTTP/2 400
content-type: application/problem+json
x-correlation-id: b6be9ddfc75544bead6ab3204a051ec6
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.1",
    "title": "Token antiforgery ausente ou inválido.",
    "status": 400,
    "detail": "Obtenha a sessão (GET /api/v2/auth/sessao) e envie o cabeçalho X-XSRF-TOKEN.",
    "instance": "POST /api/v2/auth/login",
    "traceId": "00-28f5dcc51c7b91faa2c6e186038b60dd-0153b516fa741816-00",
    "correlationId": "b6be9ddfc75544bead6ab3204a051ec6"
}

### Login como ana@portal.local
$ curl -c /tmp/tmp.qzHMclHn3e/x1 -b /tmp/tmp.qzHMclHn3e/x1 -X POST -H Content-Type: application/json -H X-XSRF-
TOKEN: <token omitido> -d {"email":"ana@portal.local","senha":"s …
HTTP/2 401
content-type: application/problem+json
x-correlation-id: c0e603c3760946b9887b1180a35aeccf
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.2",
    "title": "Não foi possível entrar.",
    "status": 401,
    "detail": "E-mail ou senha inválidos, ou acesso temporariamente bloqueado.",
    "instance": "POST /api/v2/auth/login",
    "traceId": "00-26e7fa2e444ae96a7eaf7f59b51697ba-62fe0ee59bb01d41-00",
    "correlationId": "c0e603c3760946b9887b1180a35aeccf"
}

### Login como naoexiste@portal.local
$ curl -c /tmp/tmp.qzHMclHn3e/x2 -b /tmp/tmp.qzHMclHn3e/x2 -X POST -H Content-Type: application/json -H X-XSRF-
TOKEN: <token omitido> -d {"email":"naoexiste@portal.local","sen …
HTTP/2 401
content-type: application/problem+json
x-correlation-id: 5fc7f5be794b4f13a98162b5ebfe8c8d
{
    "type": "https://tools.ietf.org/html/rfc9110#section-15.5.2",
    "title": "Não foi possível entrar.",
    "status": 401,
    "detail": "E-mail ou senha inválidos, ou acesso temporariamente bloqueado.",
    "instance": "POST /api/v2/auth/login",
    "traceId": "00-304dafcd464ac364353800fc02f4b77b-a96d601925c05cdd-00",
    "correlationId": "5fc7f5be794b4f13a98162b5ebfe8c8d"
}

### Login como ana@portal.local
$ curl -c /tmp/tmp.qzHMclHn3e/ana -b /tmp/tmp.qzHMclHn3e/ana -X POST -H Content-Type: application/json -H X-XSRF-
TOKEN: <token omitido> -d {"email":"ana@portal.local","senha": …
HTTP/2 200
content-type: application/json; charset=utf-8
cache-control: no-cache,no-store
set-cookie: portal.sessao=<valor omitido>; path=/; secure; samesite=strict; httponly
set-cookie: XSRF-TOKEN=<valor omitido>; path=/; secure; samesite=strict
x-correlation-id: b028f1acb291470595849d7c7d842382
{
    "autenticado": true,
    "usuario": "ana@portal.local",
    "equipe": "Backend",
    "papeis": [
```

```
        "Usuario"
    ],
```

Logout e reuso do cookie antigo

```
### Bruno, com sessão, lista tarefas
$ curl -b /tmp/tmp.qzHMclHn3e/bruno https://localhost:7080/api/v2/tarefas?tamanho=1
HTTP/2 200
### Logout do Bruno
$ curl -b /tmp/tmp.qzHMclHn3e/bruno -c /tmp/tmp.qzHMclHn3e/bruno -X POST -H X-XSRF-TOKEN: <token omitido>
https://localhost:7080/api/v2/auth/logout
HTTP/2 204
### Reenvio do cookie ANTIGO (copiado antes do logout)
$ curl -b /tmp/tmp.qzHMclHn3e/bruno-copia https://localhost:7080/api/v2/tarefas?tamanho=1
HTTP/2 401
```

Auditoria e usuários do seed

```
===== Auditoria de segurança (EventosSeguranca) — sem senha ou token =====
Id Evento Usuario Sucesso Detalhe OcorridoEm
-- ------ ------- ------- ------- ----------
1 Login ana@portal.local 0 senha incorreta 2026-11-09 01:45:04
2 Login naoexiste@portal.local 0 conta inexistente 2026-11-09 01:45:04
3 Login ana@portal.local 1 NULL 2026-11-09 01:45:05
4 Login bruno@portal.local 1 NULL 2026-11-09 01:45:05
5 Login carla@portal.local 1 NULL 2026-11-09 01:45:05
6 Login visitante@portal.local 1 NULL 2026-11-09 01:45:05
7 Logout bruno@portal.local 1 NULL 2026-11-09 01:45:07

===== Usuários, papéis e claims criados pelo seed controlado =====
Email Papel Claims
----- ----- ------
ana@portal.local Usuario equipe=Backend
bruno@portal.local Usuario equipe=Frontend
carla@portal.local Administrador equipe=Backend, permissao=tarefas.editar-outras-equipes
visitante@portal.local (nenhum) equipe=Geral

-- Verificação: a senha de teste aparece em alguma evidência ou log?
Não encontrada em docs/, src/, tests/, scripts/.
```

Navegação na SPA (Playwright)

```
Acesso anônimo a /tarefas/8 redirecionou para: http://localhost:5173/login
Após login, voltou para: http://localhost:5173/tarefas/8
Após recarregar (F5), sessão restaurada: ana@portal.local · Backend · Usuario
Cookies visíveis ao JavaScript: XSRF-TOKEN
Após logout, /tarefas redireciona para: http://localhost:5173/login
```



Figura B1 – Rota protegida redireciona para /login

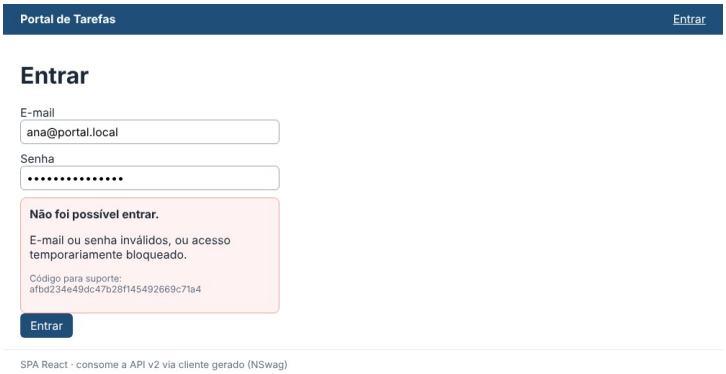


Figura B2 – Login inválido com mensagem genérica



Figura B3 – Ana na tarefa da própria equipe: pode concluir

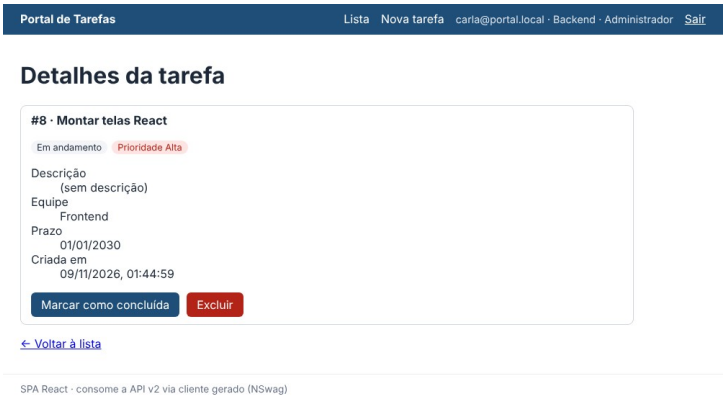


Figura B4 – Carla (Administrador + claim) em tarefa de outra equipe: concluir e excluir

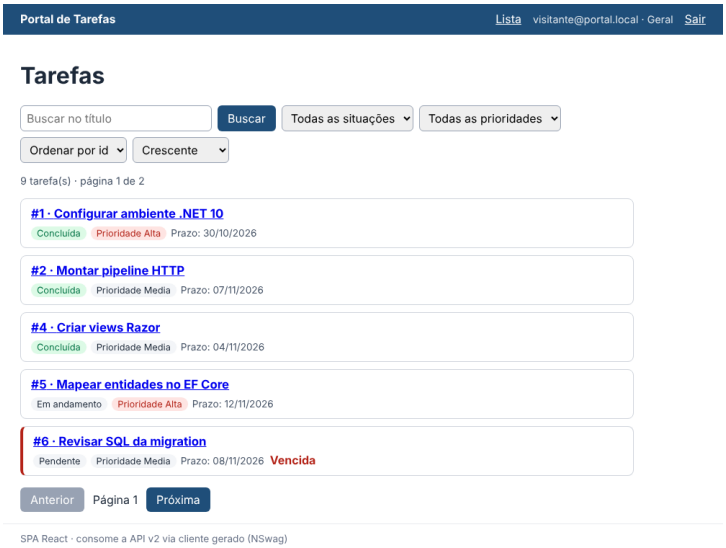


Figura B5 – Visitante: sem o link Nova tarefa

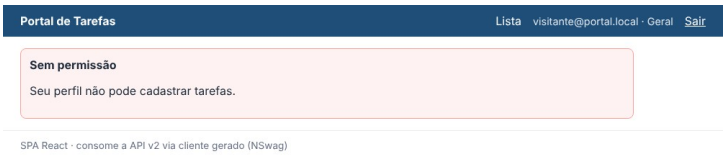


Figura B6 – Visitante acessando /tarefas/nova pela URL